



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Taller 1: Análisis léxico LEX

Álvaro Camilo Torres Rodriguez - alvtorresro@unal.edu.co

Sergio Nicolás Correa Escobar - secorreae@unal.edu.co

Juan David Cárdenas Galvis - jcardenasgal@unal.edu.co

Laura Vanesa Alvarez Lafont - lalvarezla@unal.edu.co

Nicolás Andrés Bolaños Fernandez - nbolanosf@unal.edu.co

Santiago Andrés Fetecua Pulgarín - sfetecua@unal.edu.co

Docente

Jorge Eduardo Ortiz Triviño

Universidad Nacional de Colombia

Departamento de Ingeniería de Sistemas e Industrial

Lenguajes de programación

Bogotá D.C., Colombia

2026

Contenidos

1. Marco Teórico.....	3
2. Descripción del Problema.....	7
3. Justificación del Problema.....	7
4. Diseño de Solución.....	8
5. Código Fuente.....	18
6. Manual de Usuario.....	19
a. Requisitos.....	19
b. Ejecución e Interfaz Principal.....	19
c. Gestión y Edición de Memoria RAM.....	20
d. Procesamiento de Lenguaje.....	20
e. Consideraciones.....	26
7. Manual Técnico.....	28
a. Introducción y Visión General del Sistema.....	28
b. Preprocesador.....	28
c. Ensamblador.....	29
d. Enlazador-Cargador.....	30
e. Módulo de análisis léxico del compilador.....	30
f. Conclusiones.....	31
8. Experimentación y Análisis de Resultados.....	33
Bibliografía.....	40

1. Marco Teórico

Un Sistema de Procesamiento de Lenguaje (SPL) es un conjunto de herramientas que permiten transformar un programa escrito en un lenguaje fuente en una representación ejecutable por una máquina. Este proceso involucra múltiples etapas, cada una con responsabilidades específicas dentro de la cadena de traducción.

Un SPL clásico se compone de:

- **Preprocesador**
- **Compilador**
- **Ensamblador**
- **Enlazador-Cargador**

Cada uno de estos módulos contribuye a la transformación progresiva del código, pasando de una representación de alto nivel a código ejecutable.

Preprocesador: El preprocesador es una etapa previa a la compilación encargada de realizar transformaciones textuales sobre el código fuente. No analiza la semántica del programa, sino que aplica reglas de sustitución y expansión.

Entre sus funciones principales se encuentran:

- Inclusión de archivos (`#include`)
- Definición de macros (`#define`)
- Sustitución de constantes simbólicas

El uso de herramientas como PLY (python lex and yacc) permite definir reglas basadas en expresiones regulares para reconocer y transformar estas directivas.

Compilador: El compilador es el componente central del SPL. Su función es traducir un programa en lenguaje de alto nivel a un lenguaje de nivel inferior (generalmente código intermedio o ensamblador).

Se estructura típicamente en varias fases:

Análisis Léxico: El análisis léxico es la primera fase del compilador. Se encarga de transformar una secuencia de caracteres en una secuencia de tokens.

Un token representa una unidad significativa del lenguaje, como:

- Identificadores
- Palabras reservadas

- Operadores
- Literales

Este proceso se fundamenta en el uso de expresiones regulares y autómatas finitos, que permiten reconocer patrones dentro del texto. Las categorías léxicas definen los tipos de tokens que el compilador reconoce. Algunas categorías típicas incluyen:

- Identificadores: nombres de variables, funciones
- Palabras reservadas: if, while, return
- Constantes: números enteros, reales, cadenas
- Operadores: +, -, *, /
- Delimitadores: ;, ,, {, }

Cada categoría se define mediante patrones formales, usualmente expresiones regulares. Las expresiones regulares permiten describir formalmente los patrones que conforman los tokens. Estas definiciones son la base para la construcción de analizadores léxicos automatizados.

Análisis Sintáctico

El análisis sintáctico, también conocido como parsing, es la fase del compilador encargada de verificar que la secuencia de tokens generada por el análisis léxico cumple con las reglas gramaticales del lenguaje.

Esta etapa se basa en una gramática formal, generalmente una gramática libre de contexto (GLC), que define la estructura válida de los programas. El analizador sintáctico construye una representación estructurada del programa, como:

- Árbol de derivación
- Árbol sintáctico abstracto (AST)

El análisis sintáctico permite detectar errores como: el uso incorrecto de delimitadores (paréntesis, llaves), estructuras mal formadas, o el orden inválido de los elementos del lenguaje

Esta fase es fundamental, ya que establece la estructura jerárquica del programa y sirve como base para el análisis semántico posterior.

Análisis Semántico

El análisis semántico es la fase del compilador encargada de verificar que el programa, además de ser sintácticamente correcto, tenga un significado válido dentro del contexto del lenguaje.

En esta etapa se trabaja sobre el árbol sintáctico generado previamente y se utilizan estructuras como la tabla de símbolos para validar información contextual.

Entre las principales tareas del análisis semántico se encuentran:

- Verificación de tipos (type checking)
- Validación de declaraciones de variables
- Control de alcance (scope)
- Compatibilidad entre operaciones y operandos
- Detección de usos indebidos de identificadores

Por ejemplo, el análisis semántico permite detectar errores como el uso de variables no declaradas, las asignaciones entre tipos incompatibles, y las llamadas a funciones con número incorrecto de argumentos

A diferencia del análisis léxico y sintáctico, que se basan en estructuras formales, el análisis semántico requiere conocimiento del contexto y las reglas del lenguaje, lo que lo convierte en una etapa más compleja.

Esta fase garantiza la coherencia lógica del programa antes de proceder a etapas posteriores como la generación de código.

Ensamblador: El ensamblador traduce código en lenguaje ensamblador a código máquina. Este proceso implica desde la traducción de instrucciones simbólicas a códigos binarios, pasando por la resolución de direcciones hasta el manejo de etiquetas

En el contexto del proyecto, el ensamblador se implementa también mediante PLY tanto para su lexer como para su parser, enfocándose en el reconocimiento de instrucciones y operandos.

Enlazador-Cargador: El enlazador-cargador constituye una etapa fundamental dentro del Sistema de Procesamiento de Lenguaje, encargada de integrar y preparar el programa final para su ejecución en memoria. En primer lugar, el enlazador (linker) se ocupa de combinar múltiples módulos objeto generados de manera independiente durante las fases previas de compilación o ensamblado. Este proceso implica la resolución de referencias externas, es decir, la asociación correcta entre identificadores definidos en distintos módulos, garantizando así la coherencia global del programa. Asimismo, el enlazador construye una única unidad ejecutable, organizando las distintas secciones del programa (código, datos, etc.) y estableciendo las direcciones relativas necesarias para su correcto funcionamiento.

Posteriormente, el cargador (loader) asume la responsabilidad de ubicar el programa enlazado en la memoria principal del sistema. Para ello, realiza procesos de relocalización, ajustando las direcciones de memoria según la posición real en la que el programa será cargado, y prepara el entorno necesario para su ejecución, incluyendo la inicialización de registros y estructuras auxiliares. Esta fase resulta esencial para garantizar que el programa pueda ejecutarse correctamente en el entorno destino, independientemente de las condiciones en las que fue compilado originalmente. En conjunto, el enlazador-cargador permite la modularidad, reutilización de código y correcta transición entre la representación intermedia y la ejecución efectiva del programa.

Paradigma Imperativo y Tipos de Datos Abstractos (TDA): El desarrollo del compilador propuesto en este proyecto se fundamenta en el paradigma imperativo, el cual se caracteriza por la descripción explícita de una secuencia de instrucciones que modifican el estado del sistema a lo largo del tiempo. En este enfoque, el programa se concibe como una serie de operaciones que actúan sobre variables almacenadas en memoria, utilizando estructuras de control que determinan el flujo de ejecución, tales como la secuencia, la selección (mediante construcciones condicionales como if-else) y la iteración (a través de estructuras como while o for). Este paradigma resulta particularmente adecuado para la construcción de compiladores, dado que permite un control preciso sobre el manejo de datos, la memoria y las estructuras internas necesarias para el procesamiento del lenguaje.

Adicionalmente, el diseño del sistema incorpora el uso de Tipos de Datos Abstractos (TDA), los cuales permiten modelar de manera estructurada y modular las entidades necesarias para el funcionamiento del compilador. Los TDA facilitan la encapsulación de datos junto con las operaciones que se pueden realizar sobre ellos, promoviendo la claridad, reutilización y mantenibilidad del código. En el contexto del compilador, estos pueden incluir estructuras como listas o secuencias para el manejo de tokens, pilas para la implementación de algoritmos de análisis sintáctico, y tablas de símbolos para el almacenamiento y consulta de información relevante sobre identificadores. Asimismo, el lenguaje objetivo contempla el manejo de variables, arreglos y matrices, lo que implica la necesidad de definir representaciones internas adecuadas y mecanismos para su manipulación. En conjunto, la combinación del paradigma imperativo con el uso de TDA proporciona una base sólida y organizada para la implementación eficiente y escalable del sistema de procesamiento de lenguaje.

2. Descripción del Problema

En el contexto del desarrollo de compiladores y herramientas de traducción, uno de los principales desafíos consiste en comprender e implementar de manera integrada las distintas etapas que conforman un Sistema de Procesamiento de Lenguaje (SPL). Aunque los conceptos asociados a cada componente —como el análisis léxico o el preprocesamiento— son estudiados de forma individual, existe una dificultad al momento de articularlos dentro de una arquitectura funcional que garantice un flujo correcto de transformación del programa.

El problema que aborda este taller se centra en el diseño e implementación parcial de un SPL, específicamente en sus fases iniciales, donde el código fuente es sometido a una serie de transformaciones formales antes de su compilación completa. Estas etapas incluyen el preprocesamiento del código, la traducción a un lenguaje ensamblador, la integración de módulos mediante enlazado y la preparación para su ejecución a través del cargador, así como la identificación estructurada de tokens mediante el análisis léxico.

Desde una perspectiva ingenieril, el reto consiste en modelar cada uno de estos componentes como sistemas de transformación bien definidos, con entradas, salidas y reglas de operación claramente especificadas. Esto implica el uso de herramientas formales como especificaciones basadas en expresiones regulares y el empleo de generadores de analizadores léxicos, que permitan construir soluciones deterministas, eficientes y mantenibles.

Adicionalmente, el problema involucra la definición explícita del vocabulario del lenguaje, la clasificación de sus unidades léxicas y la construcción de patrones de reconocimiento que permitan procesar correctamente el código fuente. Todo esto debe realizarse bajo un enfoque modular, asegurando que cada componente del sistema produzca resultados consistentes e interoperables con las demás etapas del SPL.

3. Justificación del Problema

La construcción de un Sistema de Procesamiento de Lenguaje en sus fases iniciales responde a la necesidad de consolidar una comprensión integral del proceso de traducción de programas, más allá del estudio aislado de sus componentes. En particular, la implementación de módulos como el preprocesador, el ensamblador, el enlazador-cargador y el analizador léxico permite entender cómo un programa fuente es transformado progresivamente mediante una cadena de procesamiento estructurada, donde cada etapa cumple un rol específico dentro de un flujo bien definido.

Desde una perspectiva técnica, este enfoque resulta fundamental, ya que obliga a modelar cada componente como un sistema de transformación formal, con especificaciones claras de entrada, salida y reglas de operación. La utilización de herramientas basadas en expresiones regulares y generadores de analizadores léxicos introduce al estudiante en prácticas propias del diseño de compiladores reales, donde la precisión en la definición del vocabulario, la correcta clasificación de tokens y la consistencia en el reconocimiento de patrones son aspectos críticos para garantizar el correcto funcionamiento del sistema.

Adicionalmente, la implementación de estos módulos permite enfrentar problemáticas propias del desarrollo de herramientas de traducción, tales como la expansión de macros, la resolución de referencias, la estructuración de información simbólica y la correcta segmentación del código fuente. Estos desafíos no solo refuerzan conceptos teóricos de lenguajes formales, sino que también desarrollan habilidades de diseño modular, abstracción y manejo de complejidad, al requerir que cada componente sea coherente tanto internamente como en su interacción con los demás.

Por otro lado, el desarrollo progresivo del SPL en fases permite construir una arquitectura escalable, en la cual las etapas implementadas en este taller constituyen la base para fases posteriores del compilador, como el análisis sintáctico, semántico y la generación de código. Este enfoque incremental facilita la validación temprana de cada componente y promueve una comprensión más profunda del flujo completo de compilación, desde el tratamiento inicial del código hasta su preparación para ejecución.

En conjunto, este ejercicio no solo cumple un propósito académico inmediato, sino que aproxima al estudiante a la ingeniería de compiladores desde una perspectiva práctica, fortaleciendo competencias clave en el diseño de sistemas, el uso de herramientas formales y la construcción de soluciones robustas dentro del ámbito del procesamiento de lenguajes.

4. Diseño de Solución

La construcción de nuestra solución se realizó de forma modular, garantizando que cada sección funcionara de forma independiente para luego integrarlas y montar su funcionamiento en el CacaoCore-64, una máquina emulada de 64 bits diseñada previamente por el mismo equipo.

Preprocesador

Se diseñó un módulo independiente basado en una arquitectura de procesamiento lineal en dos fases. En la primera fase se utiliza el lexer PLY para tokenizar el texto fuente y clasificar cada fragmento antes de interpretarlo, evitando ambigüedades entre directivas, comentarios y código arbitrario. En la segunda fase se recorren las líneas lógicas resultantes y se aplica el procesamiento correspondiente según el tipo de token: expansión de macros, resolución recursiva de inclusiones, evaluación de bloques condicionales o emisión de errores. Esta separación entre tokenización e interpretación

permite manejar correctamente casos complejos como comentarios con caracteres especiales o directivas anidadas dentro de bloques condicionales.

Para el manejo de macros se diseñó una clase dedicada (Macro) que distingue entre macros simples y macros con parámetros, aplicando la expansión mediante sustitución textual en múltiples pasadas iterativas con un límite configurable para prevenir expansión infinita. Las inclusiones de archivos se resuelven de forma recursiva manteniendo una pila de archivos activos para detectar ciclos y un conjunto de archivos ya procesados para evitar inclusiones duplicadas, con validación de rutas para prevenir accesos fuera del directorio de bibliotecas. El resultado final es una estructura PreprocessResult que separa el texto procesado del mapa de trazabilidad de líneas, desacoplando así el preprocesador del resto del compilador y permitiendo que cada componente posterior opere de forma independiente.

Compilador (Analizador léxico)

Para este módulo se empleó un paradigma orientado a objetos. La idea fue construir una clase Compiler capaz de realizar (proyectando a futuro) todas las etapas de compilación, es decir, los análisis léxico, sintáctico y semántico. Una instancia de esta clase se comunica con la interfaz encargada del SPL.

El analizador léxico, o lexer, contiene las siguientes declaraciones explícitas para el lenguaje CHOCO

Vocabulario: {let, set, func, if, otherwise, asLongAs, for, in, deliver, show, oops, int, float, text, bool, indeed, nope, nothing, mold, ohmy, summon} U Enteros U Flotantes U Identificadores U Operadores U Delimitadores U Otros

donde los patrones para los subconjuntos enunciados son:

- Enteros: `[0-9]+`
- Flotantes: `[0-9]+\.[0-9]+`
- Identificadores: `[a-zA-Z_][a-zA-Z_0-9]*`
- Operadores: `{+, -, *, /, %, ==, !=, >=, <=, <, >, =}`
- Delimitadores: `{ (, ), {, }, [, ], :, ,, . }`
- Otros: `{ whitespace, \t, \n, //}`

Categorías gramaticales: Las categorías gramaticales corresponden a la clasificación de los tokens según su función dentro de la estructura del lenguaje.

- **Palabras reservadas:** [IF, OTHERWISE, ASLONGAS, FOR, IN, LET, SET, FUNC, DELIVER, SHOW, OOPS, MOLD, OHMY, SUMMON]
- **Tipos nativos:** [INT_TYPE, FLOAT_TYPE, TEXT_TYPE, BOOL_TYPE]
- **Literales:** [NUM, STRING, INDEED, NOPE, NOTHING]
- **Identificadores:** [ID]
- **Operadores:** [PLUS, MINUS, TIMES, DIVIDE, MOD, EQ, NEG, LT, GT, LEQ, GEQ, ASSIGN]

- **Delimitadores:** [LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET, COLON, COMMA, DOT]
- **No significativos:** No tienen un token, pero son los elementos ignorados como los whitespaces, saltos de línea, y comentarios

Definiciones auxiliares: Para las posteriores definiciones de patrones

LETRA = [a-zA-Z]

DIGITO = [0-9]

GUION_BAJO = [_]

Patrones de expresiones regulares:

IF = "if"

OTHERWISE = "otherwise"

ASLONGAS = "asLongAs"

FOR = "for"

IN = "in"

LET = "let"

SET = "set"

FUNC = "func"

DELIVER = "deliver"

SHOW = "show"

OOPS = "oops"

MOLD = "mold"

OHMY = "ohmy"

SUMMON = "summon"

INT_TYPE = "int"

FLOAT_TYPE = "float"

TEXT_TYPE = "text"

BOOL_TYPE = "bool"

INDEED = "indeed"

NOPE = "nope"

NOTHING = "nothing"

NUM = DIGITO+ ("." DIGITO+)?

STRING = " \" [^\n]* \" "

ID = (LETRA | GUION_BAJO) (LETRA | DIGITO | GUION_BAJO)*

PLUS = "+"

MINUS = "-"

TIMES = "**"

DIVIDE = "/"

MOD = "%"

EQ = "=="

NEG = "!="

```
LT = "<"
GT = ">"
LEQ = "<="
GEQ = ">="
ASSIGN = "="
LPAREN = "("
RPAREN = ")"
LBRACE = "{"
RBRACE = "}"
LBRACKET = "["
RBRACKET = "]"
COLON = ":"
COMMA = ","
DOT = "."

WHITESPACE = [ \t]+
NEWLINE = \n+
COMMENT = '/*[^\\n]*'
```

Ensamblador

El módulo ensamblador fue diseñado bajo un enfoque modular y orientado a objetos, separando claramente las responsabilidades de análisis léxico, análisis sintáctico y generación de código. Su objetivo principal es traducir programas escritos en lenguaje ensamblador a una representación en código máquina compatible con la arquitectura CacaoCore-64.

La arquitectura del ensamblador se compone de tres clases principales:

AsmLexer: encargado del análisis léxico.

AsmParser: responsable del análisis sintáctico y la traducción.

Assembler: punto de entrada que coordina el proceso completo.

Análisis Léxico

El análisis léxico se implementa mediante la clase AsmLexer, utilizando la herramienta PLY. Este componente define el conjunto de tokens reconocidos por el lenguaje ensamblador, incluyendo:

Mnemónicos (MNEMONIC): instrucciones del ISA como add, mov, jmp, etc.

Registros (REGISTER): identificadores de registros (r0 a r15).

Números (NUMBER): constantes enteras.

Flotantes (FLOAT): constantes en punto flotante.

Direcciones de memoria (MEMORY): valores hexadecimales (0x...).

Variables (VAR) y etiquetas (LABEL): identificadores simbólicos.

Secciones (SECTION): .data y .text.

Símbolos estructurales: :, ,, [,], saltos de línea.

El lexer distingue entre identificadores y mnemónicos mediante una verificación contra un conjunto predefinido de instrucciones. Además, ignora comentarios y espacios en blanco, permitiendo un procesamiento limpio del código fuente.

Análisis Sintáctico y Traducción

El análisis sintáctico se implementa en la clase AsmParser, la cual define una gramática basada en reglas de producción utilizando PLY (yacc). Esta gramática permite reconocer:

Definición de etiquetas

Declaración de variables en la sección .data

Instrucciones en la sección .text

Diferentes modos de direccionamiento

El parser mantiene varias estructuras internas clave:

Tabla de símbolos (symbol_table): almacena etiquetas y sus direcciones.

Tabla de variables (var_table): asigna direcciones a variables.

Lista de instrucciones (program_text) y datos (program_data)**.

Lista de referencias pendientes (pending) para etiquetas no resueltas.

Las instrucciones son traducidas directamente a código hexadecimal utilizando la especificación del ISA (MICROINSTRUCTION_SPECS). Dependiendo del tipo de instrucción, se manejan distintos formatos, tales como:

Registro–Registro

Registro–Inmediato

Registro–Memoria

Memoria–Registro

Acceso indirecto

Salto con etiquetas

Durante la traducción, se realizan conversiones explícitas a formato binario utilizando codificación en little-endian, y se construyen las instrucciones como cadenas hexadecimales listas para ser cargadas en memoria.

Para el manejo de etiquetas, el ensamblador soporta referencias adelantadas mediante una lista de pendientes, que posteriormente se resuelven una vez finalizado el análisis completo del programa.

Generación de Salida

La clase `Assembler` actúa como punto de entrada al módulo. Esta encapsula el uso del parser y provee métodos para:

- Procesar código ensamblador de entrada.
- Resolver referencias pendientes.
- Generar la salida final en formato texto.
- Exportar el resultado a un archivo.

El resultado final consiste en una secuencia de instrucciones en formato hexadecimal, organizada según las secciones `.data` y `.text`, lista para ser cargada en memoria del emulador.

Enlazador-cargador

El módulo enlazador-cargador fue diseñado bajo un enfoque modular y orientado a objetos, separando claramente las responsabilidades de análisis léxico de módulos objeto, resolución de símbolos, validación de memoria y carga en el emulador. Su objetivo principal es tomar el código relocizable generado por el ensamblador, resolver referencias entre múltiples módulos objeto, y cargar el binario ejecutable resultante en la memoria del procesador `CacaoCore-64`.

La arquitectura del enlazador-cargador se compone de cuatro clases principales:

- `AnalizadorModulos`: encargado del análisis léxico de módulos objeto.
- `EnlazadorMejorado`: responsable del enlazado y resolución de símbolos.
- `CargadorMejorado`: encargado de la validación y carga en memoria.
- `GestorEnlazadorCargador`: punto de entrada que coordina el proceso completo.

Análisis Léxico de Módulos Objeto:

El análisis léxico se implementa mediante la clase `AnalizadorModulos`, utilizando la herramienta `PLY`. Este componente define el conjunto de tokens reconocidos por el lenguaje de módulos objeto, incluyendo:

Palabras clave (SECTION): `[MODULE, CODE, DATA, SYMBOLS, EXTERNAL]`
Identificadores (ID): `[a-zA-Z_][a-zA-Z0-9_]*` Números hexadecimales (HEX): `0x[0-9A-Fa-f]+` o `[0-9A-Fa-f]+` Números decimales (NUM): `[0-9]+` Delimitadores: `:`, `,`, `[`, `]`, `{`, `}` Símbolos especiales: `whitespace`, saltos de línea

El lexer tokeniza la estructura de módulos objeto que tiene el formato:

`[MODULE nombre_modulo]`

`[CODE] HEX HEX HEX HEX ...`

`[DATA] HEX HEX HEX HEX ...`

`[SYMBOLS] simbolo1:tipo:valor, simbolo2:tipo:valor, ...`

`[EXTERNAL] ref1:posicion:tipo, ref2:posicion:tipo, ...`

Donde:

- `nombre_modulo` es un identificador único del módulo
- `HEX` son bytes en formato hexadecimal (00-FF)
- `tipo` puede ser `"code"` o `"data"`
- `valor` es la dirección relativa al módulo (en hex con prefijo `0x` o decimal)
- `posicion` es el offset dentro del módulo donde resolver la referencia
- `ref` es el nombre del símbolo externo a resolver

Análisis Sintáctico y Enlazado

El proceso de enlazado se implementa en la clase `EnlazadorMejorado`, la cual coordina la resolución de referencias entre múltiples módulos objeto. Este componente mantiene varias estructuras internas clave:

Tabla de símbolos global (`symbol_table`): almacena todos los símbolos resueltos y sus direcciones finales.

Tabla de referencias pendientes (`pending_references`): mantiene referencias no resueltas para resolución posterior.

Gestor de módulos (`modules`): colección de módulos parseados con `AnalizadorModulos`.

Tabla de símbolos por módulo: mapeo de identificadores a direcciones relativas dentro de cada módulo.

El proceso de enlazado realiza las siguientes operaciones:

1. Carga de Módulos: Cada archivo de módulo objeto se parsea utilizando `AnalizadorModulos`, generando una estructura `ParsedModule` que contiene el código, datos y tabla de símbolos del módulo.

2. Construcción de Tabla de Símbolos Global:

- Iterar sobre todos los módulos
- Para cada símbolo interno, ajustar su valor según la dirección base asignada al módulo
- Detectar conflictos (símbolos duplicados)
- Almacenar en tabla global con dirección final resuelta

3. Resolución de Referencias Externas:

- Por cada referencia pendiente en cada módulo:
- Buscar el símbolo en la tabla global
- Si existe: calcular la dirección final y actualizar la instrucción en el módulo
- Si no existe: registrar error de símbolo no resuelto

4. Construcción de Binario Ejecutable:

- Concatenar todas las secciones CODE en orden de módulos
- Concatenar todas las secciones DATA en orden de módulos
- Incluir tabla de símbolos global

- Crear estructura BinarioEjectable con metadata

Validación y Carga en Memoria

La clase CargadorMejorado encapsula la lógica de validación de memoria e inicialización del procesador. Este componente:

Valida que la dirección base sea válida ($\geq 0x1000$ para programas de usuario) Verifica que el código quepa en la zona CODE (0x1000-0x3FFFF, 252 KB) Verifica que los datos quepan en la zona DATA (0x40000-0x7FFFF, 256 KB) Valida que no existan solapamientos entre secciones Comprueba disponibilidad de RAM en el emulador

Una vez validado el binario, el cargador:

1. Detiene el procesador temporalmente (si está activo)
2. Limpia registros temporales (MAR, MDR) para evitar conflictos
3. Escribe el código en memoria a partir de la dirección base
4. Escribe los datos en la zona DATA designada
5. Inicializa registros del procesador:
 - PC (Program Counter): dirección base
 - SP (Stack Pointer): tope de la pila (STACK_END)
 - Flags de estado: todos en cero
 - Registros acumuladores: limpios
6. Reactiva el procesador y retorna estado de éxito

Generación de Salida

La clase GestorEnlazadorCargador actúa como punto de entrada al módulo, proporcionando una interfaz simplificada que encapsula el uso del enlazador y cargador. Este componente ofrece métodos para:

Procesar múltiples archivos de módulos objeto desde disco
Procesar módulos desde contenido de texto directo
Resolver referencias y generar binario ejecutable
Cargar el binario en memoria del emulador
Recuperar tabla de símbolos global
Obtener descripciones de errores ocurridos.

El resultado final consiste en código y datos residentes en memoria del CacaoCore-64, con el procesador inicializado y listo para comenzar ejecución desde la dirección especificada. El sistema soporta múltiples módulos objeto con resolución automática de símbolos entre ellos, permitiendo compilación modular de programas complejos.

5. Código Fuente

El código fuente se encuentra adjunto en el entregable T1 Torres Rodriguez Álvaro Camilo 02.zip

6. Manual de Usuario

CacaoCore-64 es un emulador educativo de un procesador de 64 bits que implementa una arquitectura Von Neumann con un espacio de memoria RAM de 1 MB y un conjunto de instrucciones personalizado. La herramienta permite comprender el funcionamiento interno de un procesador mediante la visualización y manipulación directa de registros, memoria y flujo de ejecución. El presente manual describe los requisitos, el procedimiento de ejecución, el uso de la interfaz gráfica y las pautas básicas de operación y resolución de problemas.

a. Requisitos

Para ejecutar el emulador se requiere:

- Python 3.8 o superior.
- Tkinter (incluido por defecto en la mayoría de instalaciones de Python).
- PLY (librería que incluye los equivalentes de LEX y YACC en Python).
Opcionalmente, se puede utilizar uv sync
- Aproximadamente 10 MB de espacio en disco.
- Resolución recomendada: 1920x1080.

Para verificar Python, se puede usar el comando “python --version”.

b. Ejecución e Interfaz Principal

Desde la carpeta raíz del proyecto, después de descomprimir el archivo .zip:

```
“python -m src.cacao_gui”.
```

La interfaz se abrirá automáticamente en pantalla, para salir, cierre la ventana.

La interfaz principal constituye el núcleo de interacción con el procesador simulado. En ella se visualizan los registros fundamentales del sistema, los controles de ejecución y las opciones de visualización de datos.

En la sección superior se muestran los registros de control más relevantes: el Program Counter (PC), que indica la dirección de la siguiente instrucción a ejecutar; el Instruction Register (IR), que contiene la instrucción actualmente en proceso; y el Flags Register (FR), que almacena indicadores de estado generados por operaciones aritméticas y lógicas. Estos indicadores incluyen las banderas de interrupción (I), desbordamiento (V), acarreo (C), negativo (N) y cero (Z), las cuales permiten evaluar condiciones posteriores a una operación.

Debajo de estos registros se encuentran los 16 registros de propósito general de 64 bits. Los registros r0 a r12 pueden emplearse libremente para almacenamiento temporal de datos. El registro r13 corresponde al Stack Pointer (\$sp), que apunta al tope de la pila; r14 es el Link Register (\$lr), utilizado para almacenar direcciones de retorno en llamadas a funciones; y r15 es el Acumulador (\$acc), empleado por la unidad aritmético-lógica para almacenar resultados.

La interfaz incorpora tres controles principales de ejecución. El botón BOOT reinicia completamente el procesador, limpia los registros y posiciona el Program Counter en la dirección inicial especificada por el usuario. El botón RUN FULL ejecuta el programa de manera continua hasta encontrar una instrucción HLT o producirse un error. El botón RUN STEP permite ejecutar una única instrucción por vez, facilitando la depuración detallada del código.

El usuario debe especificar la dirección de inicio en formato hexadecimal, sin prefijo "0x". Asimismo, puede seleccionar el formato de visualización de los registros entre hexadecimal, decimal o binario. Esta selección únicamente afecta la representación visual de los datos y no altera su valor interno.

c. Gestión y Edición de Memoria RAM

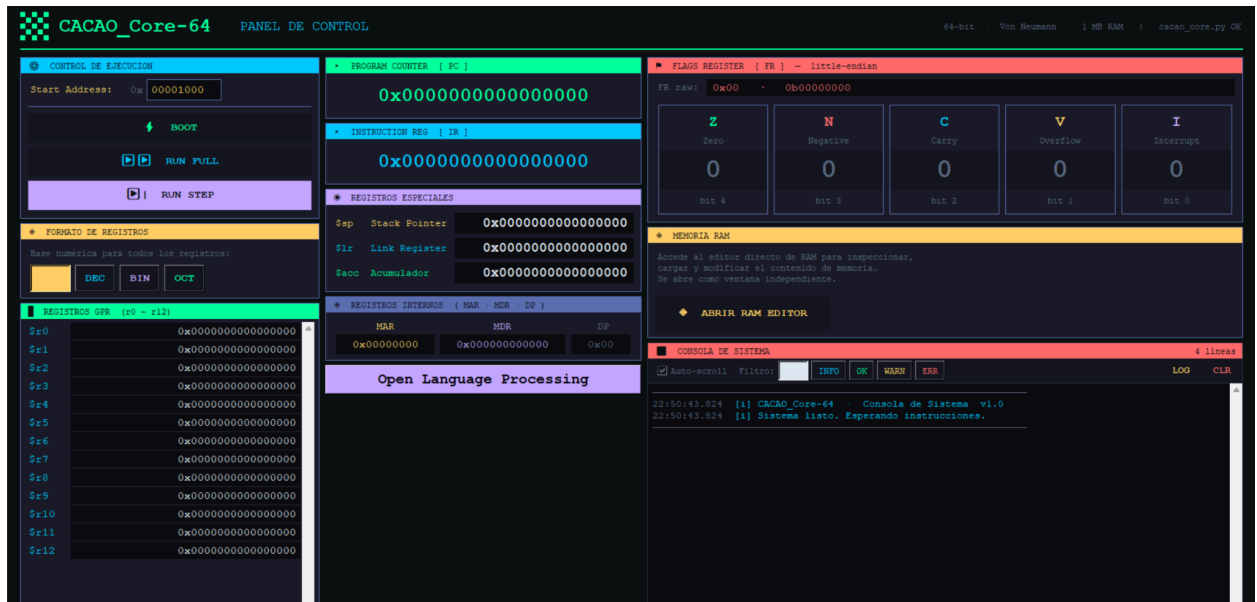
El emulador dispone de un editor de memoria RAM accesible desde la interfaz principal. La memoria total es de 1 MB y se encuentra segmentada lógicamente en zonas destinadas a sistema, código, datos estáticos, heap y stack.

La zona de código comienza en la dirección hexadecimal 0x00001000, siendo el punto habitual para cargar programas. El usuario puede introducir los bytes manualmente, cargarlos desde un archivo en formato binario o hexadecimal, o modificarlos directamente por dirección. Cada celda representa un byte expresado en dos dígitos hexadecimales.

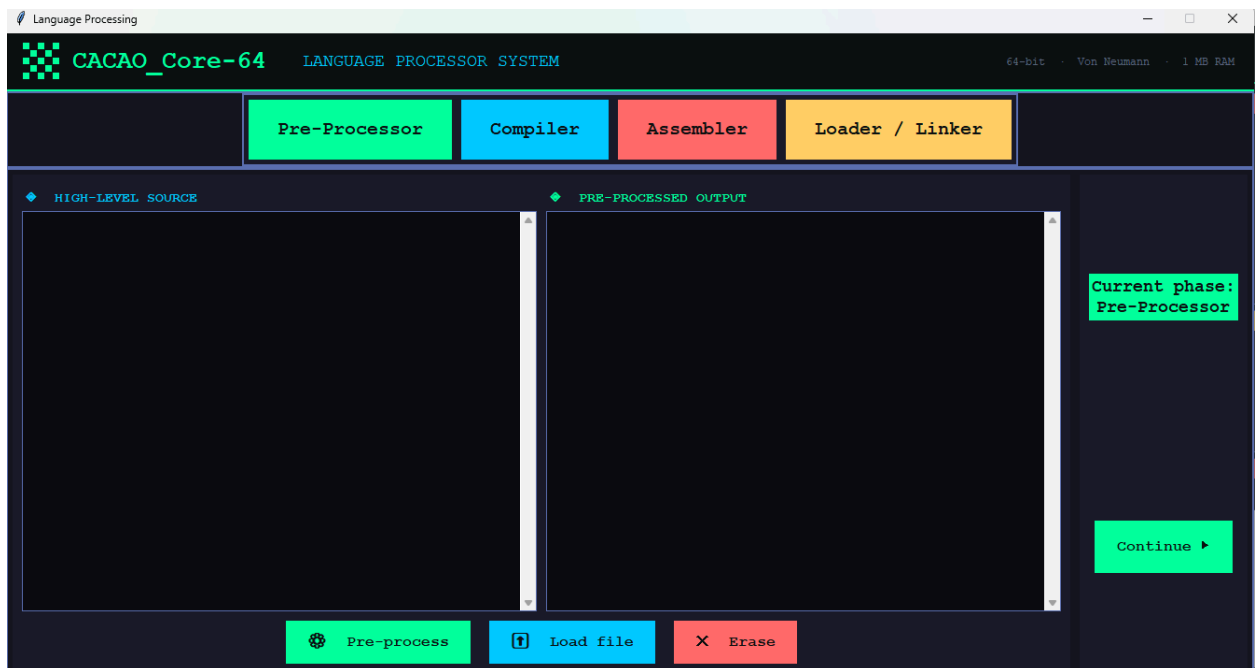
Una vez cargado el programa, la información permanece en memoria incluso después de cerrar el editor, permitiendo regresar a la interfaz principal para iniciar la ejecución.

d. Procesamiento de Lenguaje

Es en esta sección donde se implementan todas las funcionalidades requeridas para el taller 1, para acceder a esta presione el botón "Open Language Processing" que se muestra a continuación:



Posterior al click se debe de abrir la siguiente ventana auxiliar automáticamente:



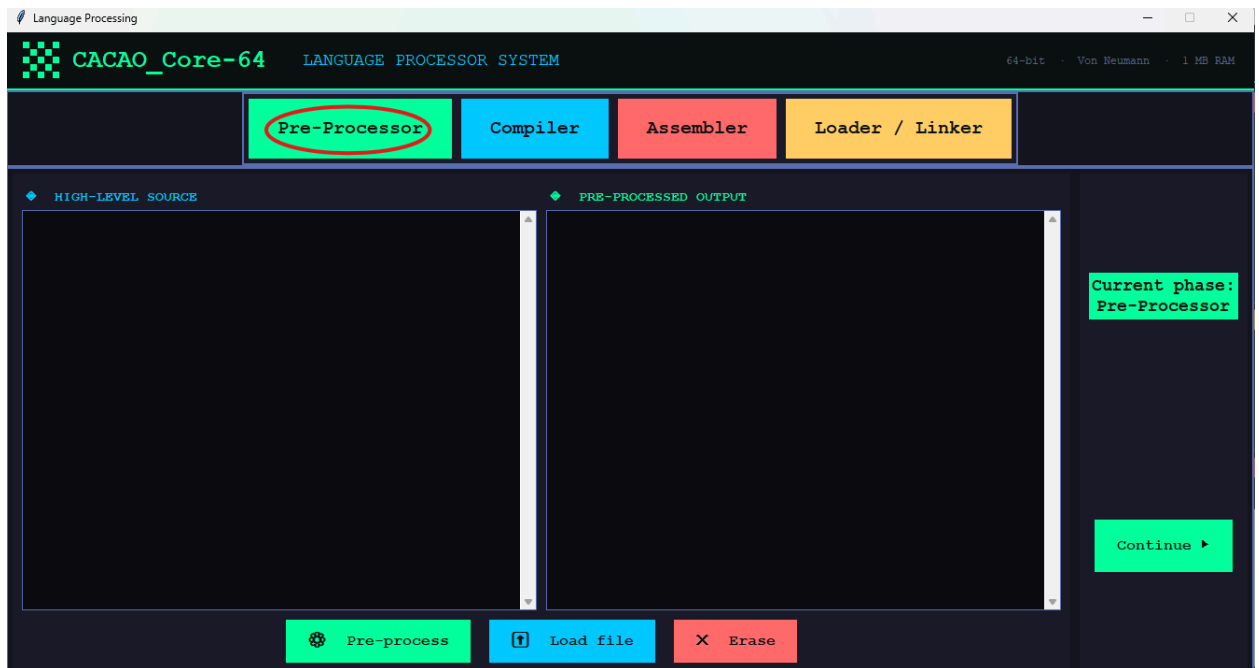
Esta cuenta con las fases propuestas para elaborar para esta tarea, que corresponden

a:

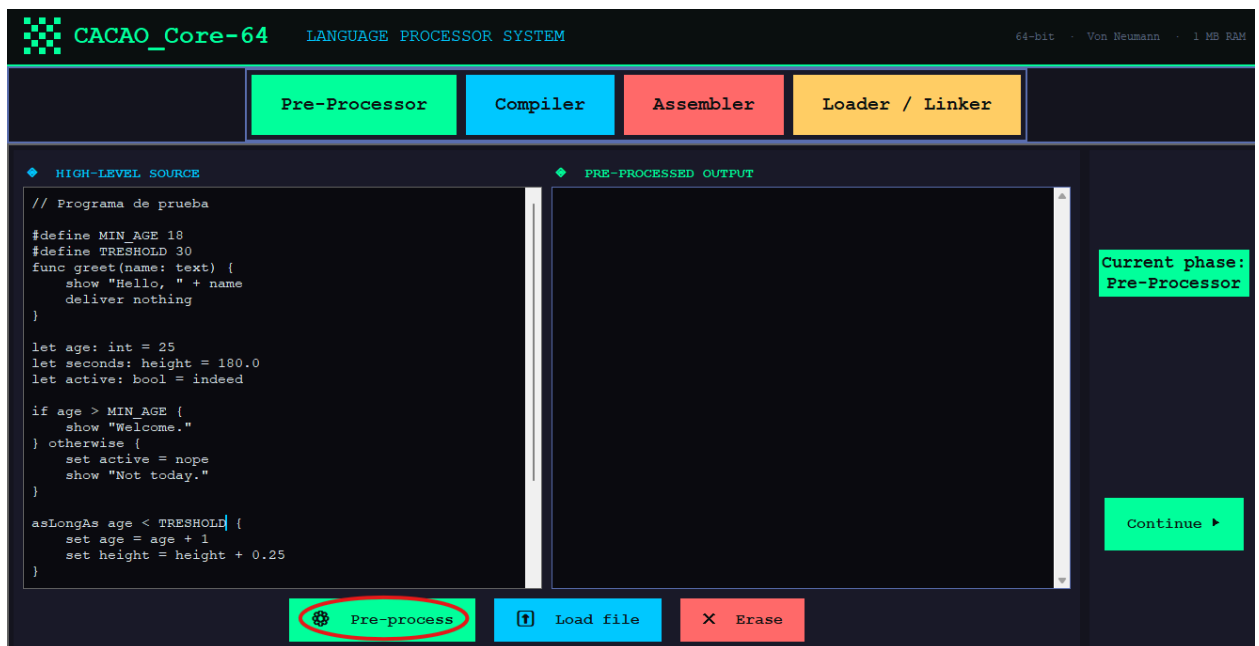
- Pre-procesador
- Compiler → Analisis Lexico
- Assembler
- Linker y Loader

A continuación abordaremos el funcionamiento de cada una de estas fases:

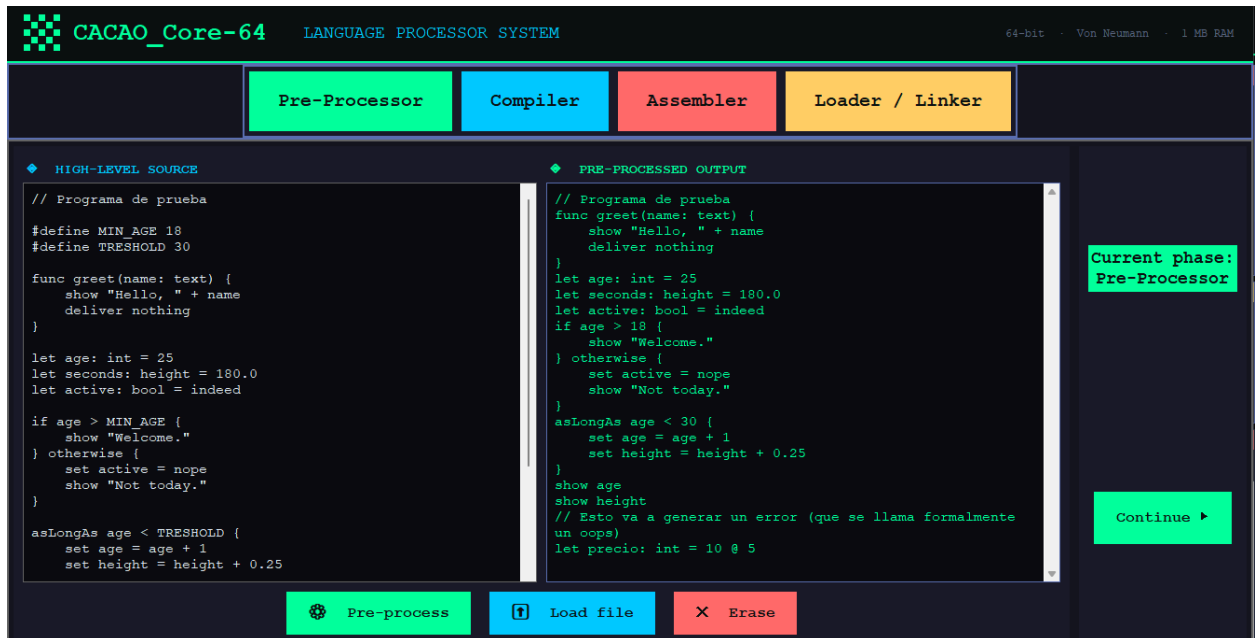
1. Pre-procesador:



Es posible acceder a esta fase presionando su respectivo botón encerrado en el círculo rojo. Para comenzar está la posibilidad de subir un archivo presionando el botón load file y escogiendo el archivo que sea de interés o si se desea escribir directamente el código en el campo:

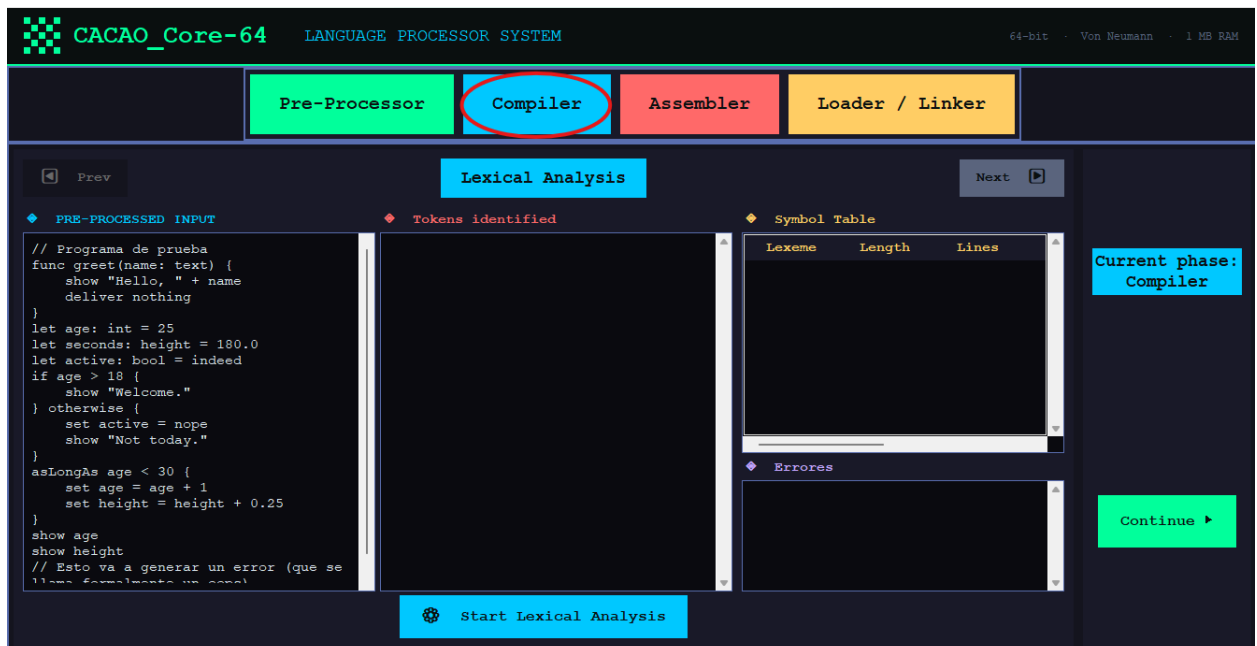


Ya con el código a procesar en el campo izquierdo se presiona el botón encerrado en la imagen para así ver la correspondiente salida pre-procesada:



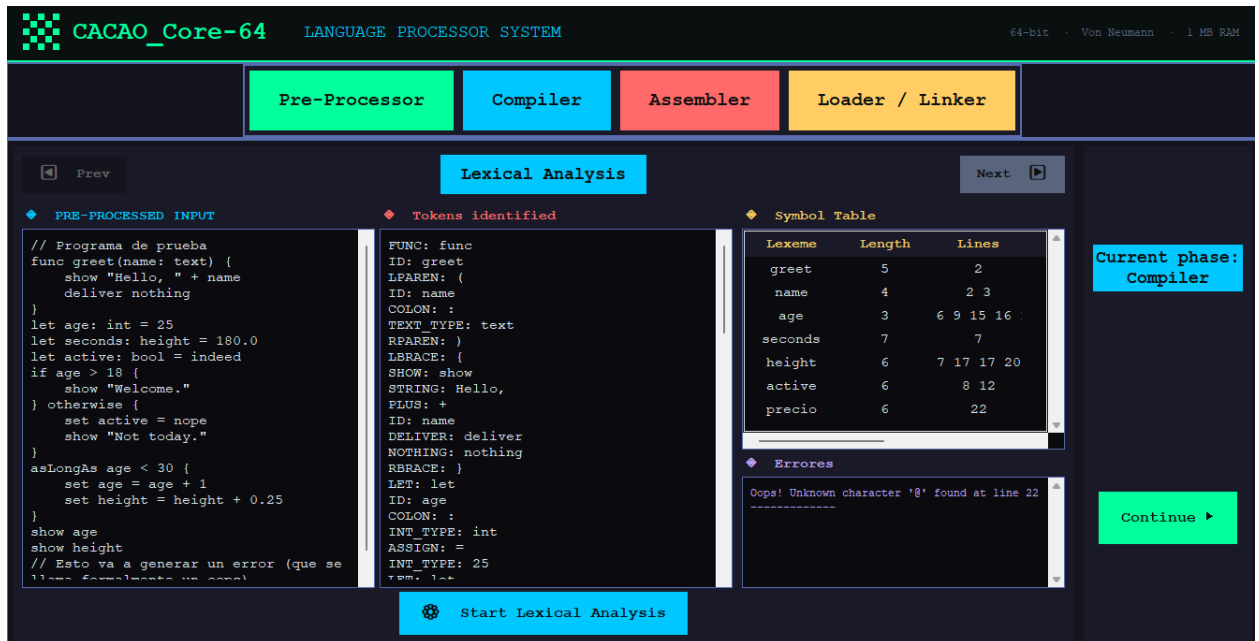
Esta salida está lista para pasar por el proceso de compilación, para acceder a la siguiente fase del procesamiento presione el botón “continue” ubicado a la derecha de la interfaz.

2. Analisis Lexico:



Esta fase es accedida ya sea mediante el botón continue desde la sección pre-processor o directamente mediante el botón encerrado “Compiler”, el campo de más a la izquierda contendrá el código fuente a compilar que podrá ser

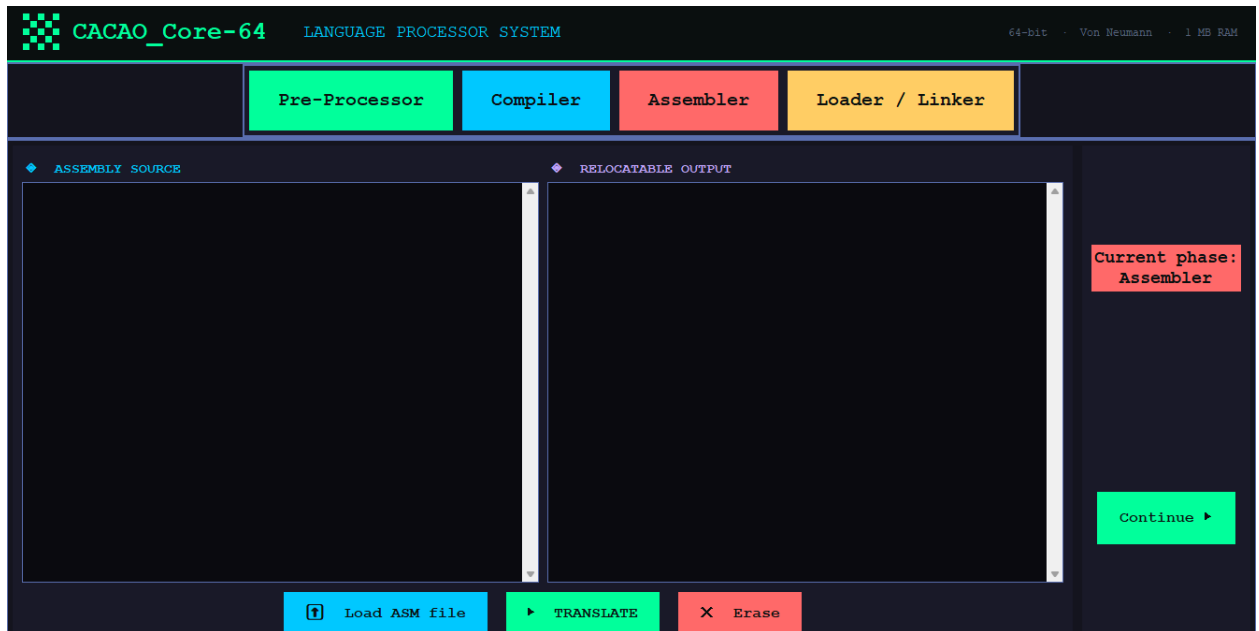
escrito directamente o puede ser producto de la fase de preprocesamiento, para empezar el análisis léxico presione con el campo input ya cargado el botón “Start Lexical Analysis”:



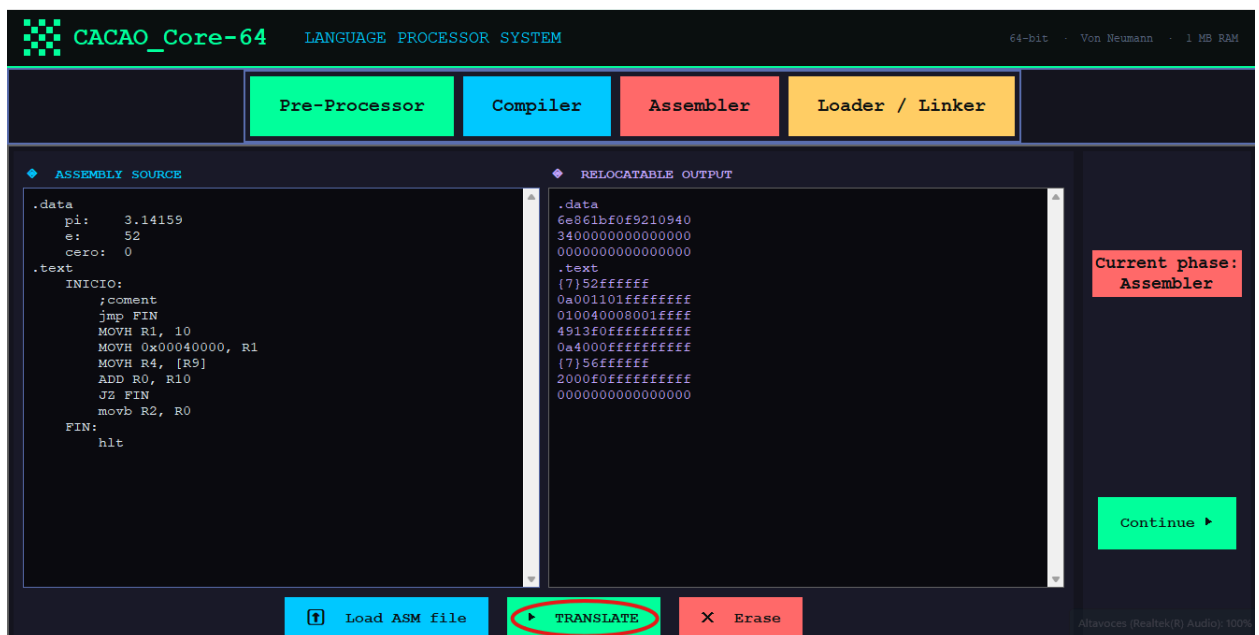
El resultado de este análisis léxico es todos los tokens encontrados en el campo tokens identified, la tabla de símbolos actualizada con los lexemas su longitud y las líneas donde aparecen y una ventana de error que informa de tokens que no pudieron ser reconocidos.

3. Assembler:

Esta fase para esta versión del proyecto ha de ser accedida mediante su botón correspondiente “Assembler”:



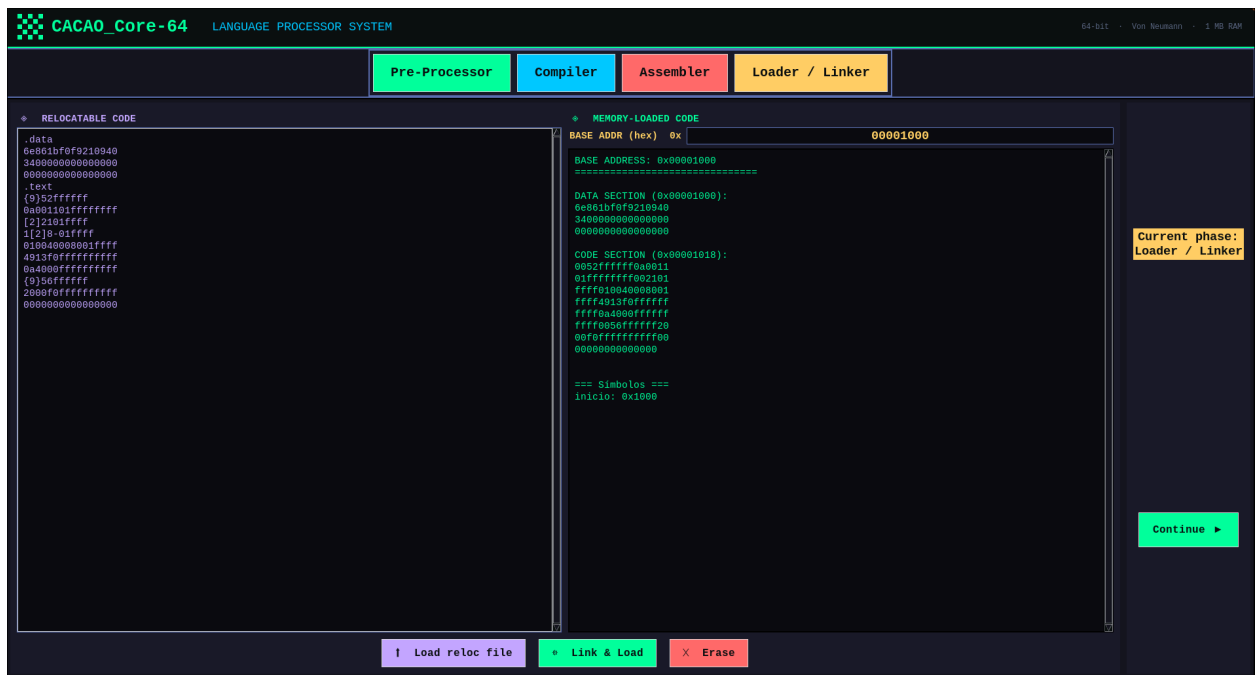
En la interfaz habrá dos campos uno con el código en asm y otro que recibirá el output de código relocalizable correspondiente, para subir un archivo a él source haga uso del botón “Load ASM file” o escriba el código asm deseado directamente:



Con el código a traducir en el campo izq presione el botón encerrado “TRANSLATE” con el cual producirá el código reloj resultante como se muestra en la imagen previa, para continuar con el flujo del procesamiento presione continúe una vez finalizada la etapa.

4. Linker y Loader:

En esta última etapa dada una dirección de memoria escogida previamente en el entry de la sección se calcularán las direcciones absolutas para todas aquellas referencias relativas y posteriormente se escribirá todo el código binario resultante en la ram desde la debida posición de memoria.



e. Consideraciones

Si el programa no avanza o parece detenido, es probable que se encuentre en un bucle infinito o que no contenga una instrucción HLT. En estos casos se recomienda reiniciar con BOOT y utilizar el modo RUN STEP para identificar el punto de bloqueo. Si el Program Counter no cambia tras ejecutar una instrucción, debe verificarse que el sistema haya sido reiniciado correctamente y que la dirección de inicio sea válida y esté en formato hexadecimal.

Cuando los registros presentan valores inesperados, la estrategia adecuada consiste en ejecutar el programa paso a paso para determinar qué instrucción genera la modificación no deseada. También es recomendable revisar las direcciones de memoria utilizadas. Los errores de tipo "Address Out of Range" indican que el programa intenta acceder a direcciones fuera del rango válido de la RAM (0x00000000 a 0x000FFFFF). Deben revisarse los operandos y cálculos de dirección.

Si el campo de dirección inicial no acepta la entrada, debe comprobarse que únicamente se utilicen caracteres hexadecimales válidos (0–9 y A–F) sin prefijo adicional. Finalmente, si la interfaz gráfica presenta problemas de visualización, se recomienda ajustar la resolución del sistema o revisar la configuración de escalado del sistema operativo.

CacaoCore-64 está diseñado como herramienta académica para el estudio de arquitectura de computadores. El uso del modo paso a paso, la observación sistemática de registros y flags, y la verificación periódica de la memoria constituyen buenas prácticas para la comprensión profunda del comportamiento del procesador.

Este entorno permite experimentar de manera controlada con conceptos fundamentales como ejecución secuencial, manejo de pila, direccionamiento de memoria y control de flujo.

7. Manual Técnico

El presente documento describe con mayor profundidad el diseño e implementación de los componentes léxicos del SPL del emulador CacaoCore-64. Está dirigido a desarrolladores que requieran comprender el diseño del sistema, extender su funcionalidad o realizar procesos de depuración a nivel de código fuente y utilización avanzada.

a. Introducción y Visión General del Sistema

El presente documento describe de manera formal el análisis, diseño e implementación del Sistema de Procesamiento del Lenguaje SPL, desarrollado para la arquitectura CacaoCore-64 de 64 bits. Este sistema constituye una cadena completa de traducción que permite transformar programas escritos en el lenguaje SPL en código ejecutable, listo para ser cargado y ejecutado en memoria.

El sistema se ha concebido bajo un enfoque modular y multifase, en el cual cada componente cumple una función específica dentro del proceso de compilación y ejecución. En el contexto del Taller #01, se abordan cuatro componentes fundamentales: el preprocesador, el ensamblador, el enlazador-cargador y el módulo de análisis léxico del compilador. Cada uno de estos ha sido diseñado siguiendo principios clásicos de construcción de compiladores y empleando herramientas basadas en el metalenguaje FLEX, implementadas mediante PLY en Python.

El flujo general del sistema inicia con el código fuente SPL, pasa por una etapa de preprocesamiento donde se resuelven macros e inclusiones, continúa con el análisis léxico que transforma el código en una secuencia de tokens, y posteriormente atraviesa fases de análisis sintáctico y semántico. Finalmente, el código es traducido a ensamblador, convertido a código objeto por el ensamblador, enlazado con otros módulos y cargado en memoria para su ejecución por el procesador CacaoCore-64.

b. Preprocesador

El preprocesador constituye la primera etapa del sistema de traducción. Su propósito es preparar el código fuente antes de su análisis formal, permitiendo la expansión de macros y la inclusión de archivos externos. En este proyecto, el preprocesador ha sido diseñado e implementado utilizando herramientas equivalentes al metalenguaje FLEX, específicamente mediante PLY, lo cual permite definir reglas léxicas de manera estructurada.

Desde el punto de vista del análisis, se identificó la necesidad de soportar al menos dos directivas fundamentales: `#include` y `#define`. Estas directivas son esenciales para la

modularidad del código y la reutilización de fragmentos. El diseño del preprocesador contempla un mecanismo de lectura secuencial del código fuente, acompañado de una tabla de macros y un sistema de resolución de rutas para archivos incluidos.

En términos de implementación, el preprocesador es capaz de gestionar la directiva `#include` mediante la lectura del archivo especificado e inserción de su contenido en el flujo de entrada. Este proceso incluye el manejo de errores, como la inexistencia del archivo o problemas de acceso. Por otro lado, la directiva `#define` permite la definición de macros de sustitución textual, las cuales son almacenadas en una estructura de datos tipo diccionario y aplicadas durante el recorrido del código.

Un aspecto relevante del diseño es el mantenimiento de un mapeo de líneas de origen, lo cual permite asociar cada línea del código preprocesado con su ubicación original. Esto resulta fundamental para la generación de mensajes de error precisos en etapas posteriores del compilador, aunque aún resulta un poco difuso dada la incompletitud del compilador.

En conjunto, el preprocesador implementado cumple con los requerimientos establecidos, al gestionar correctamente macros de inclusión y definición, y proporcionar una salida coherente para las siguientes fases del sistema.

c. Ensamblador

El ensamblador es el componente encargado de traducir código en lenguaje ensamblador a código objeto en formato binario. Su diseño sigue el modelo clásico de dos etapas: análisis léxico y análisis sintáctico, ambos definidos mediante especificaciones compatibles con el metalenguaje FLEX.

Durante la fase de análisis, se identificaron los elementos fundamentales del lenguaje ensamblador de la arquitectura CacaoCore-64, tales como mnemónicos de instrucciones, registros, literales numéricos, etiquetas, directivas de sección y delimitadores. A partir de esta identificación, se definió un conjunto de tokens que permite representar formalmente cualquier programa ensamblador válido.

El diseño del ensamblador contempla un analizador léxico que reconoce estos tokens mediante expresiones regulares, seguido de un analizador sintáctico que valida la estructura de las instrucciones y construye representaciones intermedias. Adicionalmente, se incluye un mecanismo de resolución de etiquetas, que permite asignar direcciones a símbolos definidos dentro del código.

En la etapa de implementación, el ensamblador procesa el código línea por línea, construye una tabla de símbolos local y genera el código objeto correspondiente. Este código se organiza en secciones de texto y datos, y se almacena en un formato binario adecuado para su posterior procesamiento por el enlazador, cuyo paso a módulo objeto aún tiene mejoras conceptuales y de implementación a realizar.

De esta manera, el ensamblador cumple con su función de traducir correctamente el lenguaje ensamblador a un formato ejecutable intermedio, manteniendo coherencia con la arquitectura objetivo.

d. Enlazador-Cargador

El enlazador-cargador representa la etapa final del proceso de traducción. Su función principal es integrar múltiples módulos objeto en un único binario ejecutable y posteriormente cargar dicho binario en memoria para su ejecución.

Desde la perspectiva de análisis, se identificaron dos responsabilidades claramente diferenciadas: el enlazado y el cargado. El enlazado implica la resolución de referencias externas entre módulos, mientras que el cargado consiste en la ubicación del código y los datos en memoria, así como la inicialización del entorno de ejecución.

El diseño del enlazador incluye la construcción de una tabla global de símbolos, obtenida a partir de la combinación de las tablas locales de cada módulo. Esta tabla permite detectar errores como símbolos duplicados o no definidos, y facilita la asignación de direcciones finales a cada referencia. Asimismo, se implementa un proceso de reubicación, mediante el cual se ajustan las direcciones de memoria en función de la posición final del código.

En cuanto al cargador, su diseño contempla la validación de los límites de memoria, la copia de las secciones de código y datos en las direcciones correspondientes, y la inicialización de registros clave del procesador, como el contador de programa y el puntero de pila.

La implementación de este componente garantiza que los programas enlazados puedan ejecutarse correctamente en la arquitectura CacaoCore-64, cerrando así el ciclo completo de traducción.

e. Módulo de análisis léxico del compilador

El módulo de análisis léxico constituye la primera fase del compilador y tiene como objetivo transformar el código fuente en una secuencia de tokens. Este módulo ha sido diseñado e implementado utilizando especificaciones basadas en el metalenguaje FLEX, lo que permite definir de manera formal los patrones de reconocimiento del lenguaje.

i. Vocabulario del compilador

Como parte del análisis, se determinó el vocabulario completo que reconoce el compilador. Este incluye palabras clave del lenguaje (como estructuras de control, definiciones y operaciones), tipos de datos primitivos, identificadores, operadores aritméticos y relacionales, delimitadores y literales.

Las palabras clave abarcan construcciones como declaración de variables, funciones, condicionales y ciclos. Los tipos de datos incluyen enteros, flotantes, cadenas y booleanos. Los identificadores siguen reglas formales de formación, mientras que los operadores y delimitadores permiten estructurar expresiones y sentencias.

Asimismo, se reconocen literales numéricos, cadenas de texto y valores booleanos, así como comentarios que son ignorados durante el análisis.

ii. Categorías gramaticales

A partir del vocabulario definido, se establecieron las categorías gramaticales necesarias para la construcción del compilador.

Las categorías léxicas corresponden a los tokens generados por el analizador léxico, tales como identificadores, literales, operadores y palabras clave.

Esta separación permite construir una gramática formal del lenguaje SPL, la cual es utilizada posteriormente en el análisis sintáctico para generar el árbol de sintaxis abstracta.

iii. Especificación de patrones léxicos

En la etapa de diseño, se definieron los patrones de reconocimiento de tokens mediante expresiones regulares. Estas expresiones permiten identificar de manera precisa cada categoría léxica.

Por ejemplo, los identificadores se definen mediante una expresión que exige una letra o guion bajo inicial, seguida de caracteres alfanuméricos. Los números enteros y flotantes se reconocen mediante patrones numéricos específicos, mientras que las cadenas de texto se delimitan por comillas. Los operadores y delimitadores se identifican mediante expresiones simples, teniendo en cuenta la precedencia en el caso de operadores compuestos.

En la implementación, estos patrones se traducen en reglas léxicas dentro del analizador, permitiendo que el sistema procese el código fuente de manera eficiente y estructurada. Adicionalmente, se incorporan mecanismos para ignorar espacios en blanco y comentarios, así como para llevar un control del número de línea.

f. Conclusiones

El desarrollo presentado en este documento demuestra la implementación de los componentes fundamentales de un sistema de compilación completo. En particular, se ha logrado el diseño e implementación del preprocesador, el ensamblador, el enlazador-cargador y el módulo de análisis léxico del compilador, todos ellos basados en principios formales y herramientas equivalentes al metalenguaje FLEX.

Cada uno de estos componentes cumple un rol específico dentro del sistema, y su integración permite transformar programas escritos en SPL en código ejecutable para la arquitectura CacaoCore-64. El diseño modular adoptado facilita la extensión futura del sistema, especialmente en las fases de análisis sintáctico, análisis semántico y generación de código.

Este trabajo sienta las bases para el desarrollo completo del compilador en fases posteriores, en las cuales se profundizará en la validación sintáctica, semántica, en la optimización y en la generación de código eficiente.

8. Experimentación y Análisis de Resultados

Para probar la máquina se ejecutaron dos archivos uno para cada parte, el primero dedicado únicamente al flujo de proceso a lexic análisis y el otro al de assembler hasta loader and linker, los archivos usados se muestran a continuación:

Flujo preprocessor - Lexical Analysis

None

```
// Programa de prueba

#define MIN_AGE 18
#define TRESHOLD 30

func greet(name: text) {
  show "Hello, " + name
  deliver nothing
}

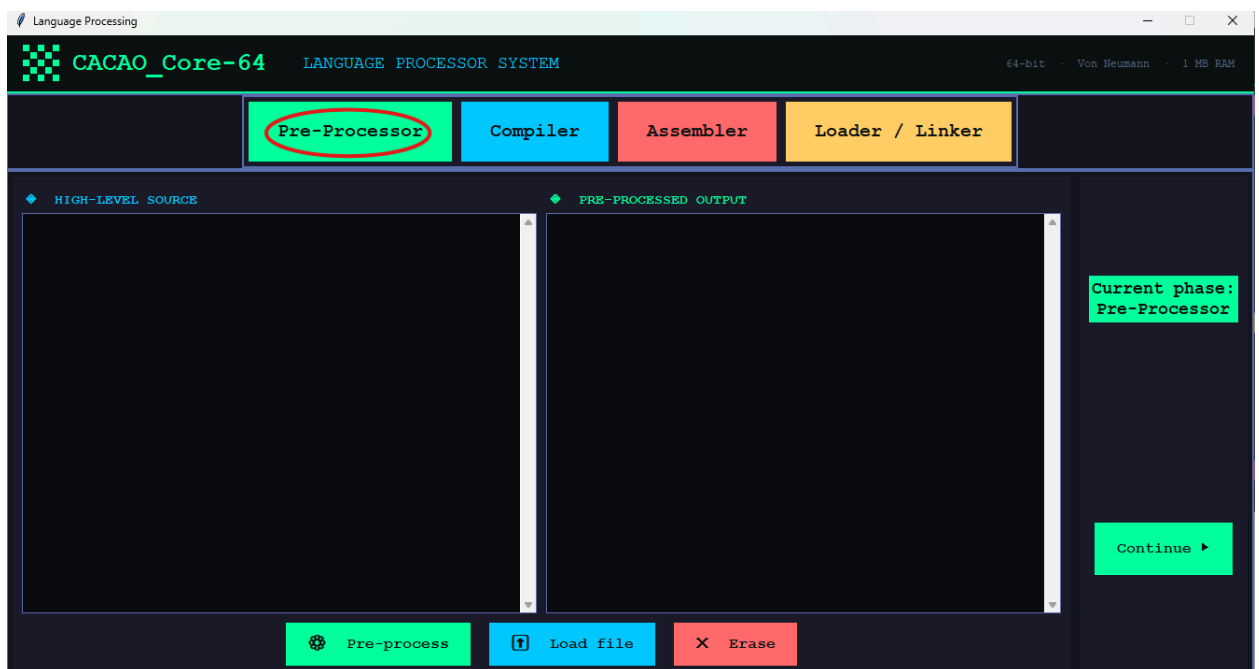
let age: int = 25
let seconds: height = 180.0
let active: bool = indeed

if age > MIN_AGE {
  show "Welcome."
} otherwise {
  set active = nope
  show "Not today."
}

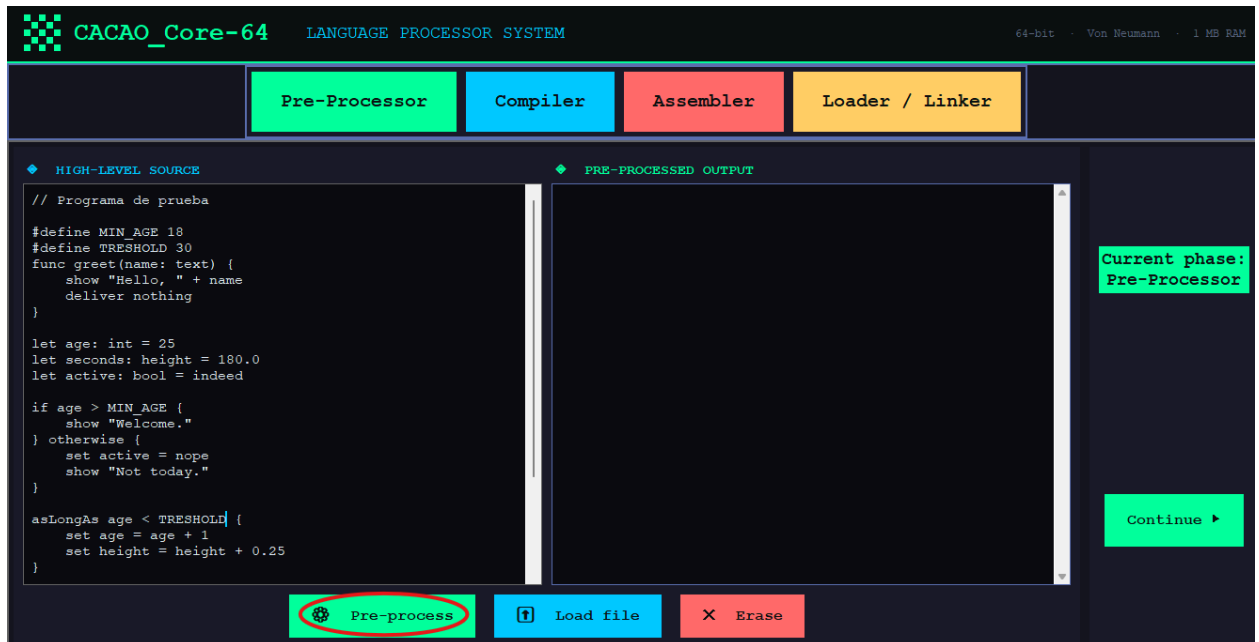
asLongAs age < TRESHOLD {
  set age = age + 1
  set height = height + 0.25
}
```

```
show age  
show height
```

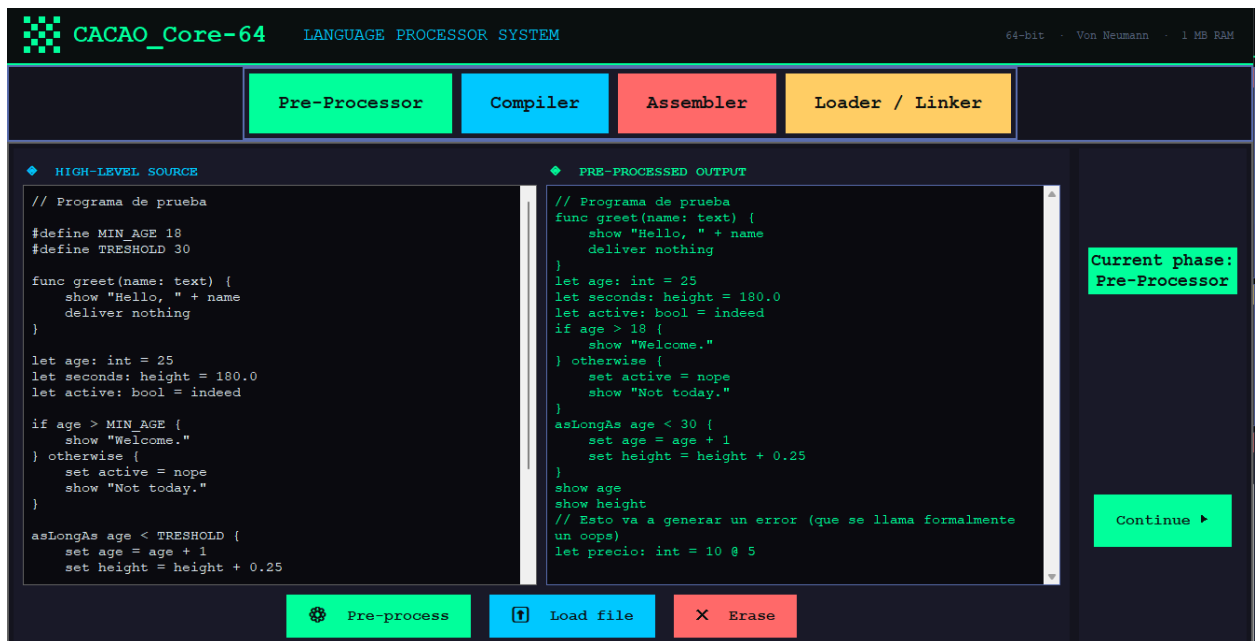
```
// Esto va a generar un error (que se llama formalmente un oops)  
let precio: int = 10 @ 5
```



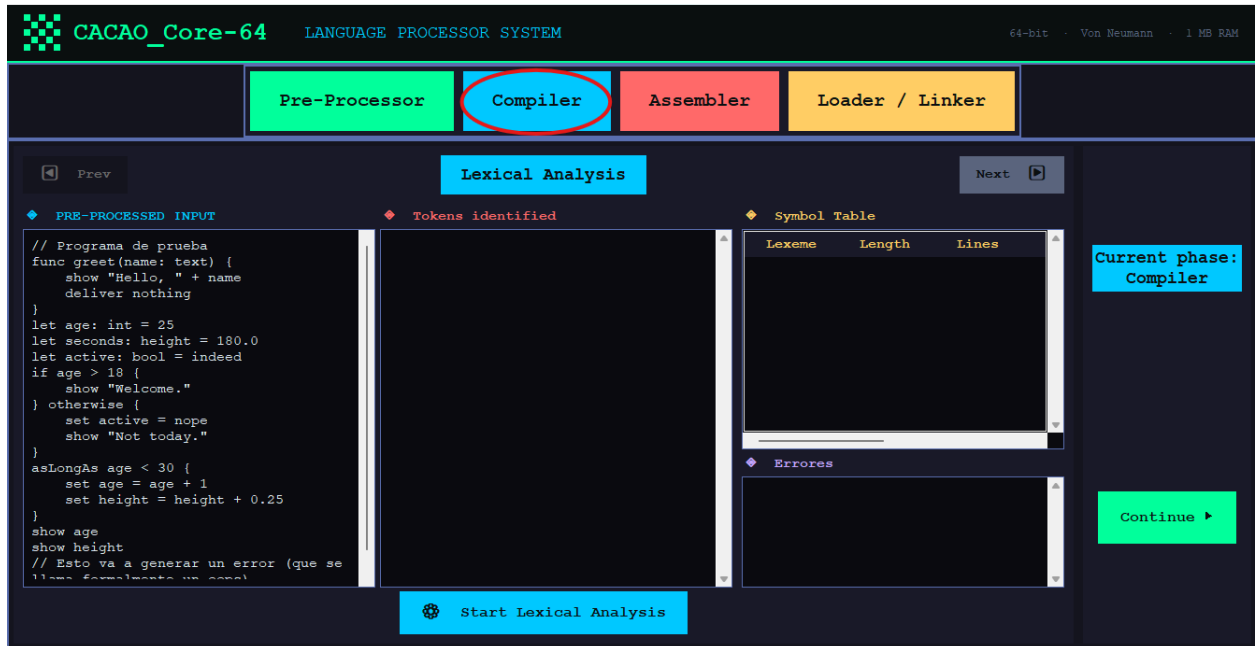
Para comenzar se sube el archivo presionando el botón load file y escogiendo el archivo que sea de interés o si se desea escribir directamente el código en el campo:



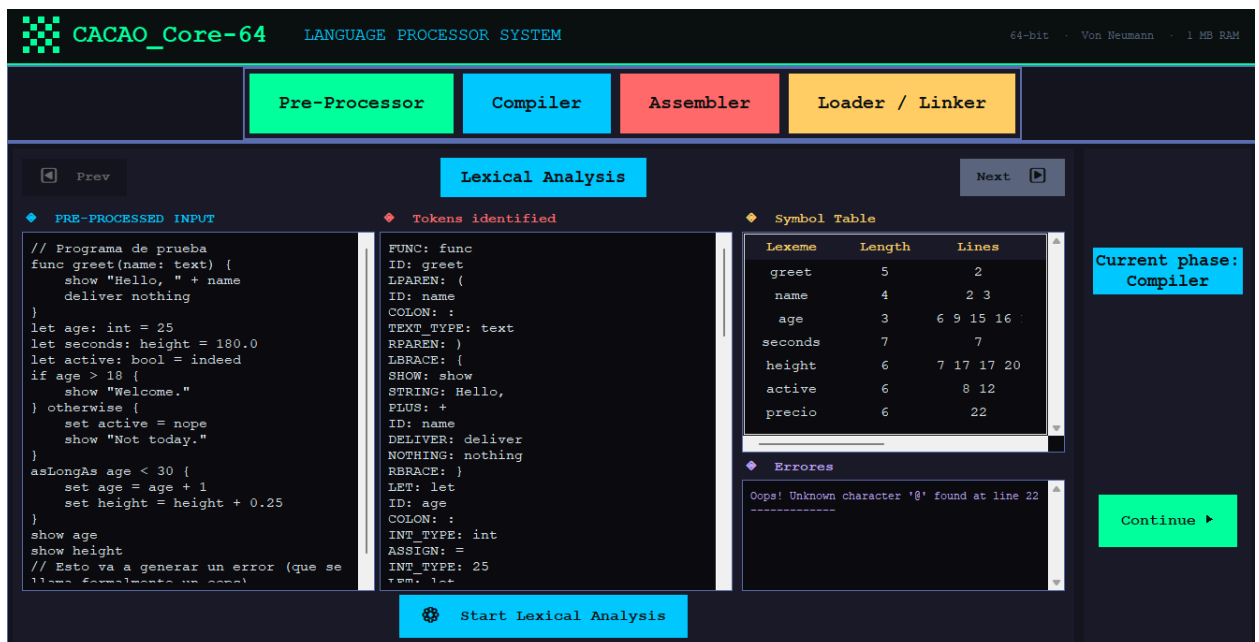
Ya con el código a procesar en el campo izquierdo se presiona el botón encerrado en la imagen para así ver la correspondiente salida pre-procesada:



Esta salida está lista para pasar por el proceso de compilación, para acceder a la siguiente fase del procesamiento presione el botón "continue" ubicado a la derecha de la interfaz. Como se puede observar los **defines** fueron reemplazados correctamente en el código de salida



En esta segunda fase el campo de más a la izquierda contiene el código fuente a compilar producto de la fase de preprocesamiento, para empezar el análisis léxico se presiona con el campo input ya cargado el botón “Start Lexical Analysis”:



El resultado de este análisis léxico es todos los tokens encontrados en el campo tokens identified, la tabla de símbolos actualizada con los lexemas, su longitud y

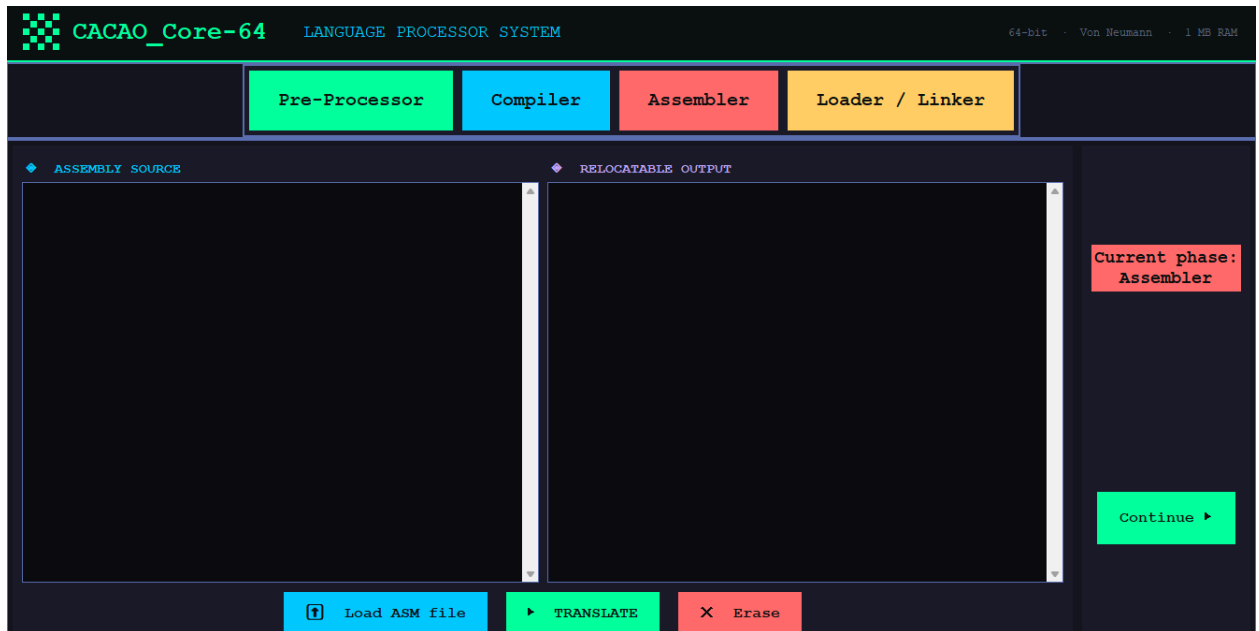
las líneas donde aparecen y una ventana de error que informa de tokens que no pudieron ser reconocidos.

Flujo assembler - Loader / Linker

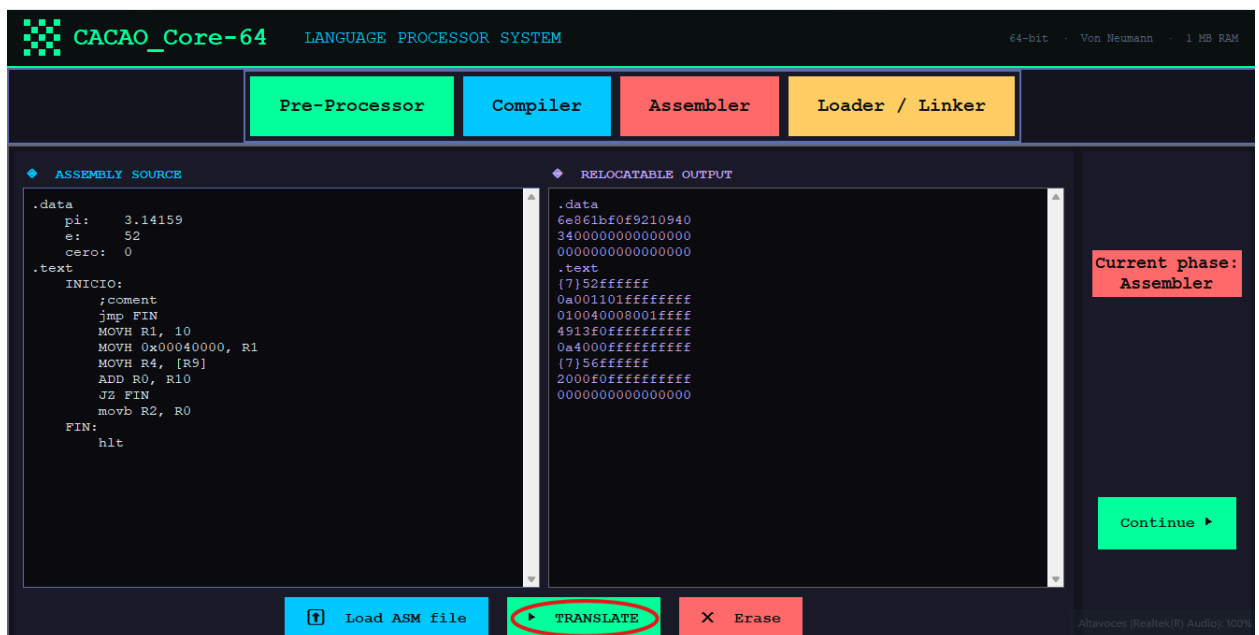
None

```
.data
pi:  3.14159
e:   52
cero: 0
.text
INICIO:
    ;coment
    jmp FIN
    MOVH R1, 10
    MOVH R1, [cero]
    MOVH [cero], R1
    MOVH 0x00040000, R1
    MOVH R4, [R9]
    ADD R0, R10
    JZ FIN
    movb R2, R0
FIN:
    hlt
```

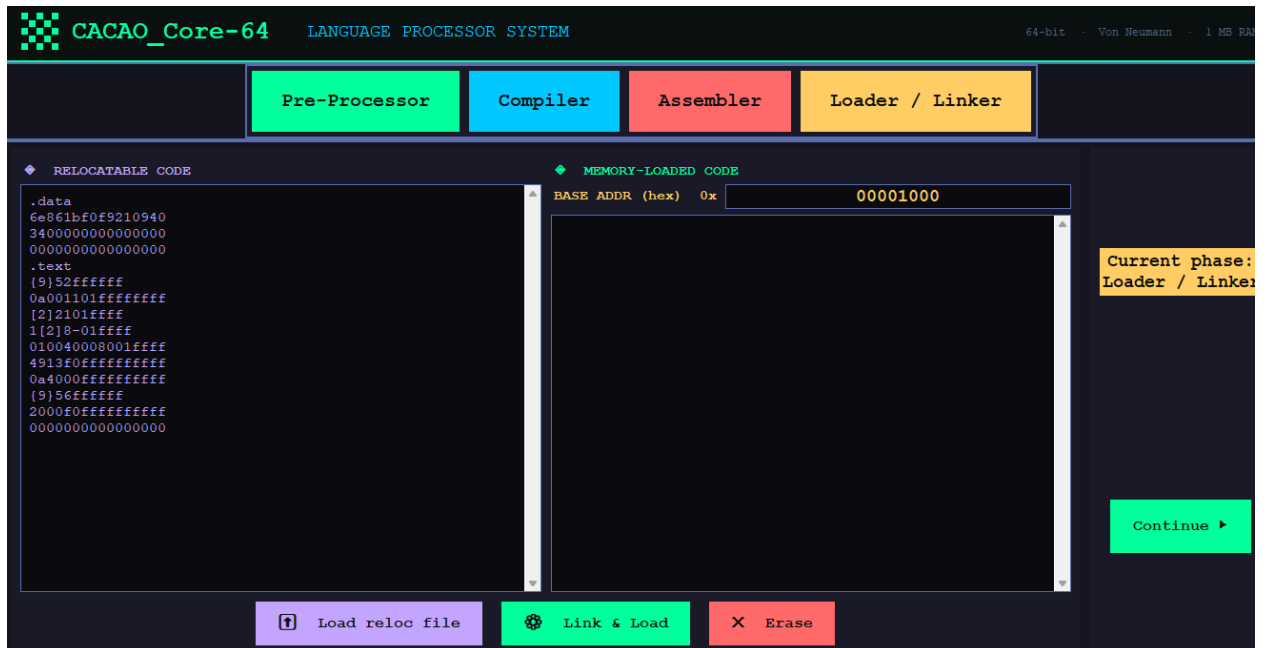
Esta fase para esta versión del proyecto ha de ser accedida mediante su botón correspondiente “Assembler”:



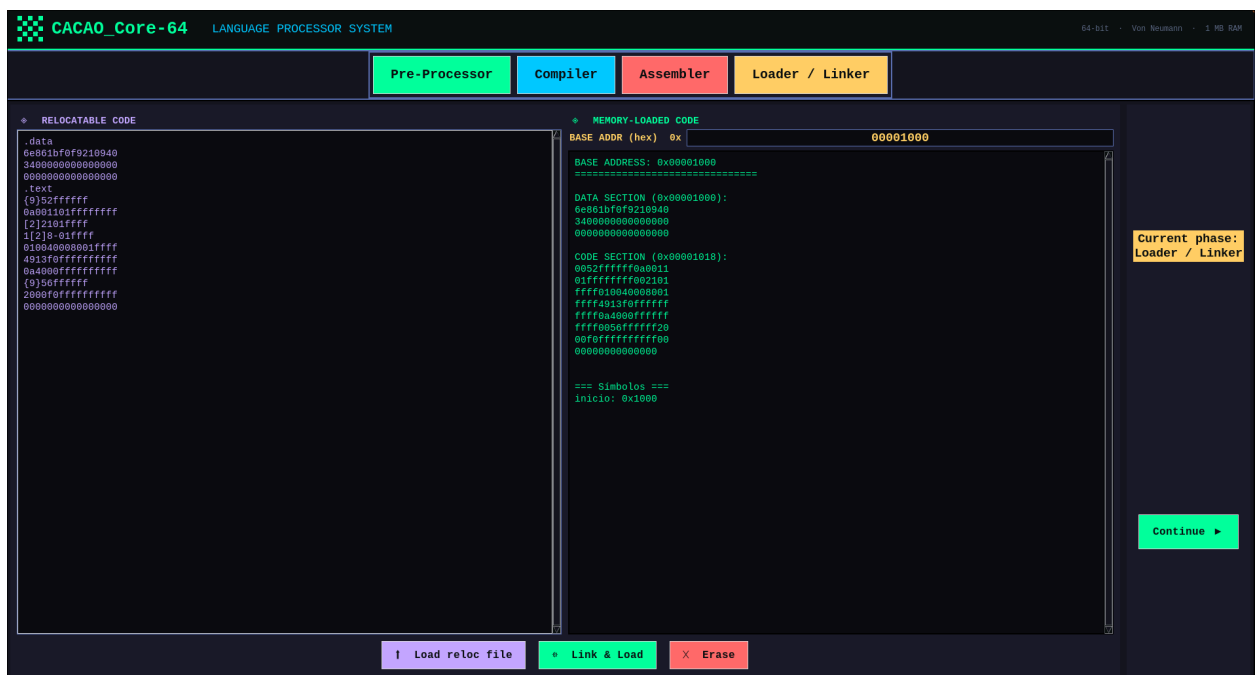
En la interfaz habrá dos campos uno con el código en asm y otro que recibirá el output de código relocalizable correspondiente, para subir un archivo a él source haga uso del botón “Load ASM file” o escriba el código asm deseado directamente:



Con el código a traducir en el campo izq presione el botón encerrado “TRANSLATE” con el cual producirá el código reloj resultante como se muestra en la imagen previa, para continuar con el flujo del procesamiento presione continue una vez finalizada la etapa.



En esta última etapa dada una dirección de memoria escogida previamente en el entry de la sección se calcularán las direcciones absolutas para todas aquellas referencias relativas y posteriormente se escribirá todo el código binario resultante en la ram desde la debida posición de memoria.



Bibliografía

- Compilers: Principles, Techniques, and Tools. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- PLY (Python Lex-Yacc). D. Beazley, *PLY (Python Lex-Yacc)*. Disponible en: <http://www.dabeaz.com/ply>